```
    /* structure to save the key and value */
    struct nlist {
      struct nlist *next; /* next entry in chain */
      char *key; /* defined key */
      char *value; /* replacement text which holds value */
    }

    /* pointer table to save list of structures */
    static struct nlist *hashtab[HASHSIZE]

    /* hash: form hash value for string s which is key */
    unsigned hash(char *s) {
      /* unsigned value ensures non negative */
      unsigned hashval;

      /* adds each character's value of the string to form
scrambled combination of the previous ones */
      for (hashval =0; *s != '\0'; s++) {
          hashval = *s + 31 * hashval;
      }

/* modulo ensures the index is within the array size */
      return hashval % HASHSIZE
    }

    /* lookup: look for s in the hash table */
    struct nlist *lookup(char *s) {
      struct nlist *np;

      for (np = hashtab[hash[(s)]; np != NULL; np = np-
>next) {
          if(strcmp(s, np->key) == 0) {
      return np;
          }
      }

      /* key not found in hash table */
      return NULL;
    }

    /* insert: add the key and value to the hash table */
    struct nlist *insert(char *key, char *value) {
      struct nlist *np;
      unsigned hashval;

      /* call the lookup function to validate if already
key exists */
/* if not exists allocate the memory for the new structure */
      if ((np = lookup(key)) == NULL) {
          np = (struct nlist *) malloc(sizeof*np));

      /* if unable to allocate memory for the structure */
          /* or unable to copy key in the struct key */
/* return NULL as it no more memory could be allocated */
          if (np == NULL || (np->key = strdup(key)) == NULL)
{
              return NULL;
          }

/* get the index for the key, by calling the hash function */
          hashval = hash(key);

          /* point the newly created structure's next to the
existing index value's struct */
          np->next = hashtab[hashval];

/* point the current pointer to the head of the index */
/* so the newly created pointer will be pointed as first */
          hashtab[hashval] = np;
      } else {
          /* if already the key is available, free up its
value so we can override below */
          free((void *) np->value);
      }

      /* save the value to the key's structure */
      if((np->value = strdup(value)) == NULL) {
          return NULL;
      }
    }
```

Note: The chaining mechanism is used above to avoid hash collisions.

Applications

Now that we have discussed hash and its implementation, let's view its common applications. In cryptography, hash functions are used to produce the hashed output from the input, which is impossible to reverse. Password verification is another important use of cryptographic hash functions. When a password is entered by the client, it is hashed and sent to the server. As the server already knows the hash of the password, it can simply compare it. If there is a middle man attack, the password is still unknown to the hacker. The Rabin Karp string search algorithm uses hashing to find a set of patterns in the string. Its real world application is to detect plagiarism. END

By: Thangaselvi Arichandrapandian

The author works in a leading bank as AVP. She is a perennial learner and loves the quote: "The more I learn, the more I realise how much I don't know." - *Albert Einstein.*